# ContractLens Milestone 2 Architecture Report

Milestone 2 — Agentic NDA Review with Retrieval-Augmented Traceability

Milestone 2

ContractLens / ContractNLI Project

April 29, 2026


Team Attribution: ContractLens Milestone Team

| Name | ID |
|---|---|
| Daniel Ashraf Ekdawi | 55-1598 |
| Mostafa Ashraf Ezzat | 55-5414 |
| Ahmed Hatim Backar | 55-24857 |
| David George Atef | 55-26933 |
| Youssef Mohamed Malek | 55-24816 |



**Executive Summary.** This report presents the Milestone 2 architecture of ContractLens, extending a batch legal-NLI baseline into an agentic, conversational NDA review system. The architecture combines orchestration, retrieval support, and schema-governed traceability while maintaining strict legal grounding constraints. A central design principle is preserved throughout: retrieval examples inform reasoning, but final legal evidence must come exclusively from the analyzed contract.

# Contents


1 Introduction / Project Context 2

2 System Overview and Design Rationale 2

3 Orchestrator 4

4 NLI Core 4

5 End-to-End Data Flow 5

6 Vector RAG 5

7 GraphRAG 6

8 Conversation Agent 7

9 Updated Playbook 7

10 Updated RunTrace Schema 8

11 Validation and Safety Constraints 9

12 Limitations and Future Work 9

13 Implemented vs Architectural / MS3 10

14 Conclusion 10

**Abstract.** This report documents the Milestone 2 architecture of ContractLens, an NDA review system built on ContractNLI that evolves from batch legal-NLI evaluation into an agentic and conversational workflow. The design combines `Qwen3-4B` orchestration, a fine-tuned `Qwen3-1.7B` NLI core, retrieval support through Vector RAG and GraphRAG, hypothesis-level decomposition, and schema-driven auditability through updated playbook metadata and `RunTrace 2.0`. The report emphasizes a core safety principle: retrieved training examples are external reasoning memory only, while final legal evidence must be grounded exclusively in spans from the analyzed contract.

## 1. Introduction / Project Context

ContractLens is an NDA review system built around the ContractNLI task setting, where fixed legal hypotheses are evaluated against contract text with evidence grounding. In Milestone 1, the project delivered a batch legal-NLI baseline that focused on per-contract hypothesis classification and trace generation in a mostly linear pipeline.

Milestone 2 extends that baseline into an agentic and conversational architecture. The system is designed to support user-driven contract questioning while preserving legal grounding constraints and auditable decision traces.

The main Milestone 2 additions are: `Qwen3-4B` orchestration, vector and graph retrieval branches, conversation memory, playbook metadata for per-hypothesis agent attribution, and `RunTrace 2.0` auditability features that separate `external_memory` from `contract_evidence`.

## 2. System Overview and Design Rationale

Milestone 1 was a batch legal-NLI baseline: a single fine-tuned model processed one contract and produced hypothesis labels/evidence in a single pipeline. Milestone 2 evolves this into an agentic NDA review architecture with explicit decomposition, retrieval-assisted reasoning, and traceable component boundaries.

The target execution pattern is **broadcast** → **fan-out** → **aggregate**: the NLI core performs a single broad pass, task-specific hypothesis work is decomposed into units, and a final aggregation stage materializes a unified trace artifact.

This decomposition improves traceability (per-hypothesis provenance and metadata), modularity (independent RAG branches and workers), and maintainability (clear separation of orchestration, retrieval, inference, and aggregation concerns). Table 1 summarizes the primary components and their current status.

As shown in Figure 1, orchestration and retrieval are separated from evidence-grounding logic, reducing the risk of retrieval leakage into final legal assertions.

This architectural framing leads directly to the orchestrator's role as routing and control logic, which coordinates model inference, retrieval, and output structuring.

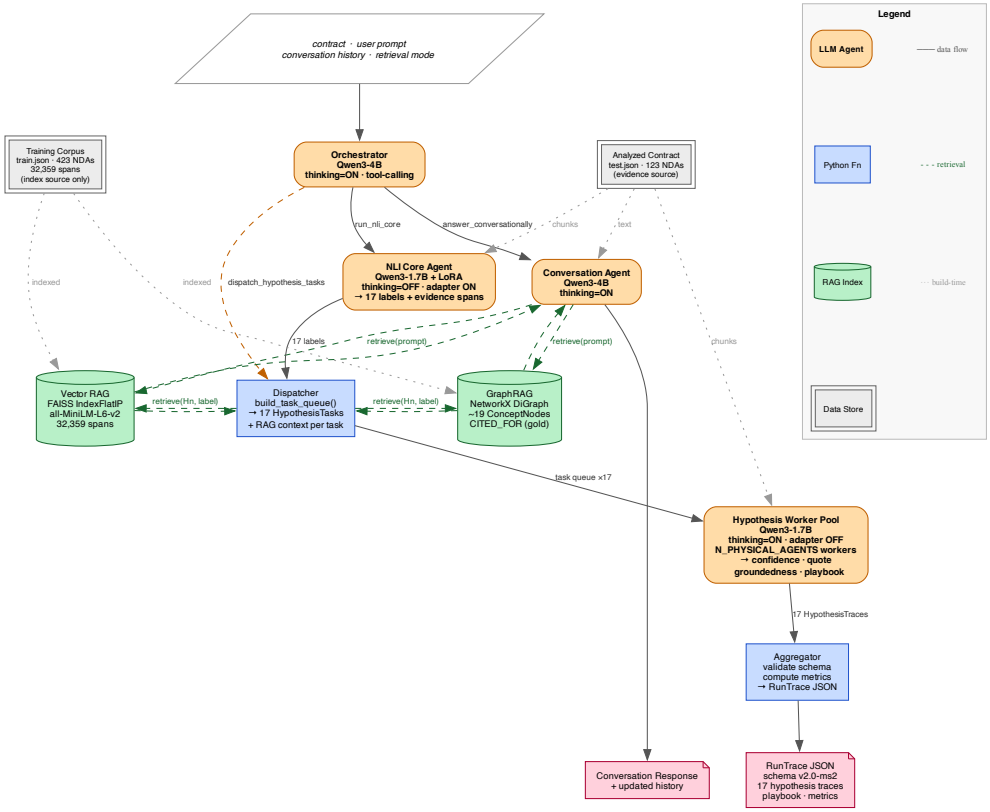


Figure 1: ContractLens Milestone 2 architecture overview

Table 1: Component Summary

| Component | Model / Method | Responsibility | Implementation Status |
|---|---|---|---|
| Orchestrator | `Qwen3-4B` | Session-level routing, tool-calling, path selection between conversation and analytical flow. | Architectural / MS3-target |
| NLI Core | Fine-tuned `Qwen3-1.7B` | Single-pass evaluation of all 17 hypotheses with structured outputs and evidence spans. | Present |
| Vector RAG | Embedding + FAISS | Semantic retrieval from `data/train.json` for reasoning support. | Present (`src/rag_vector.py`) |
| GraphRAG | Concept graph retrieval | Concept- and hypothesis-targeted retrieval over graph schema. | Present (`src/rag_graph.py`) |
| Conversation Agent | `Qwen3-4B` + history | User-facing Q&A, context injection, conversation continuity. | Present (`src/conversation_agent.py`, `agent.py`) |
| Playbook | Schema-governed ruleset | Policy mapping from hypothesis outcomes to severity/action/rationale. | Present (schema updated) |
| `RunTrace 2.0` | Schema-governed trace model | Auditable run output with retrieval/evidence separation and agent metadata. | Present (schema updated) |

## 3. Orchestrator

The orchestrator is specified as `Qwen3-4B` with thinking enabled. It is the tool-calling controller that routes execution across conversation handling and analytical tools.

Architecture specifies JSON-style function calling with call shape: `{"name": "<tool_name>", "arguments": {...}}`

Planned orchestration routes include:

- Conversation path: user prompt + history + contract + retrieval context to conversational answer.
- Analytical path: NLI core to dispatcher/task fan-out to hypothesis workers to aggregation.
- Retrieval routing: selects exactly one branch per run (`vector` or `graph`) and passes mode through downstream traces.

The orchestrator is architectural in `architecture/architecture.yaml` (MS3-target end-to-end control loop).

Because orchestration quality depends on stable structured outputs from the legal classifier, the NLI core is constrained to a dedicated model mode and output discipline.

## 4. NLI Core

The NLI core is the fine-tuned `Qwen3-1.7B` adapter model:

```
Youssef-Malek/
contractnli-vast-ai-qwen3-1.7b
```

It is designed to run once per contract over all 17 hypotheses.

Key policy:

- **Thinking OFF** for the fine-tuned NLI core (`enable_thinking=False`).
- Rationale: the adapter was trained for direct structured classification output; enabling thinking introduces format drift risk.

Expected NLI core output is hypothesis-level classification with contract-grounded evidence span indices (ENTAILED, CONTRADICTED, NOT_MENTIONED + evidence spans).

After NLI classification, retrieval is used to support reasoning and calibration, but retrieval output remains separate from final legal evidence.

## 5. End-to-End Data Flow

The intended Milestone 2 end-to-end flow is presented below. The sequence clarifies the boundary between reasoning support and admissible legal grounding:

1. The user submits a contract and a question.
2. The conversation agent loads the analyzed contract.
3. The orchestrator selects either vector retrieval or graph retrieval.
4. The selected retrieval module fetches examples from `data/train.json` only.
5. Retrieved examples are injected as `external_memory`, not final evidence.
6. The fine-tuned `Qwen3-1.7B` NLI core evaluates all 17 hypotheses.
7. Hypothesis tasks are enriched with playbook decisions and metadata.
8. The aggregator merges outputs into `RunTrace 2.0`.
9. The conversation agent returns a grounded answer to the user.

A central rule applies across all steps: final legal evidence must come from spans in the analyzed contract, never from retrieved training examples.

## 6. Vector RAG

Vector RAG is defined to index and retrieve from `data/train.json` only.

Design details:

- Embedding model: `sentence-transformers/all-MiniLM-L6-v2`.
- Similarity index: FAISS `IndexFlatIP`.
- Vectors are L2-normalized so inner product approximates cosine similarity.
- Supports both free-text retrieval and hypothesis-targeted retrieval (with optional label-aware filtering/re-ranking).

`data/test.json` is evaluation-only and must not be indexed.

Vector retrieval provides high-coverage semantic matching, while the graph branch complements it with more explicit concept-level interpretability.

## 7. GraphRAG

GraphRAG is defined with three node types:

- `SpanNode`
- `ConceptNode`
- `HypothesisNode`

And three edge types:

- `CONTAINS`
- `INVOLVES`
- `CITED_FOR`

The design uses hardcoded legal concept vocabularies and synonyms, intentionally avoiding external NLP/NER dependencies. Retrieval supports concept-based free-text graph traversal and hypothesis-targeted lookup paths. The full `ConceptNode` vocabulary consists of 19 canonical legal terms, each matched via simple substring search against lowercased span text — no spaCy, no transformer-based NER required beyond what the rest of the pipeline already loads.

| Canonical Name | Display | Example Synonyms |
|---|---|---|
| `receiving_party` | Receiving Party | receiving party, recipient, receiver |
| `disclosing_party` | Disclosing Party | disclosing party, discloser, provider |
| `confidential_information` | Confidential Information | proprietary information, trade secret, know-how |
| `employees` | Employees | personnel, staff, representatives, officers |
| `third_party` | Third Party | consultant, advisor, agent, contractor, vendor |
| `non_disclosure` | Non-Disclosure Obligation | shall not disclose, keep confidential, not reveal |
| `notice_obligation` | Notice Obligation | shall notify, give notice, written notice |
| `return_or_destroy` | Return or Destroy | return, destroy, destruction, delete, erase |
| `purpose_limitation` | Purpose Limitation | solely for, only for, permitted purpose |
| `no_license` | No License Grant | no license, no right, does not grant, no ownership |
| `reverse_engineering` | Reverse Engineering | reverse engineer, decompile, disassemble |
| `copying` | Copying | copy, copies, reproduce, duplicate |
| `independent_development` | Independent Development | independently develop, without reference to |
| `third_party_acquisition` | Third-Party Acquisition | received from, lawfully received, publicly available |
| `identification` | Identification / Marking | marked, labeled, marked confidential, stamped |
| `survival` | Survival | survive, shall survive, notwithstanding termination |
| `retention` | Retention | retain, keep, archival copy |
| `termination` | Termination | termination, expiration, end of agreement |
| `legal_compulsion` | Legal / Judicial Compulsion | required by law, court order, subpoena |
| `solicitation` | Solicitation | solicit, hire, recruit, induce, poach |

Table 2: GraphRAG `ConceptNode` vocabulary — 20 canonical legal concepts matched by substring against span text.

With retrieval context prepared, the conversation layer can answer user queries while enforcing grounding and provenance constraints.

## 8. Conversation Agent

The conversation agent is user-facing and uses `Qwen3-4B` with thinking enabled. In the repository, `src/conversation_agent.py` and `agent.py` implement the CLI interaction loop and persist history in `conversation_history.json`.

Behavioral constraints:

- Retrieval context is injected as **external memory** only.
- Retrieved training examples must never be cited as final legal evidence.
- Final legal conclusions and quotes must be grounded in the currently analyzed contract.

The implementation enforces this distinction in prompt instructions and structured trace design.

The schema updates in Milestone 2 formalize these runtime requirements into explicit, auditable data structures.

## 9. Updated Playbook

Milestone 2 updates the playbook schema while preserving Milestone 1 legal logic:

- Added `schema_version: "2.0-ms2"`
- Added per-check `agent_id` (which hypothesis agent H01–H17 produced the result)
- Added per-check `conversation_turn` (nullable integer — null for batch pipeline, non-null if triggered via conversation agent)

These additions provide the metadata needed for agentic execution and traceability without changing the underlying legal risk semantics from Milestone 1.

Illustrative MS2 playbook shape:

```
{
  "schema_version": "2.0-ms2",
  "checks": [
    {
      "hypothesis_id": "H06",
      "agent_id": "H06",
      "conversation_turn": null,
      "criticality": "P0",
      "overrides": { "NOT_MENTIONED": { "severity": "HIGH", "action": "
         CLARIFY" } },
      "rationale_templates": { "satisfied": "...", "conflict": "...", "
         missing": "..." }
    }
  ]
}
```

Note: `retrieval_mode` is a runtime parameter recorded in the `RunTrace`, not in the playbook. The playbook is a static policy file whose legal risk rules do not change between runs.

Unchanged elements:

- 17 hypothesis rules (H01-H17)
- Criticality tiers
- Override mappings
- Rationale templates
- Label/severity/action logic

## 10. Updated RunTrace Schema

Milestone 2 extends `RunTrace 2.0` to capture agentic execution context:

- Added `schema_version: "2.0-ms2"`
- Added `retrieval_mode`
- Added `conversation_history`
- Added `retrieval_context`
- Added per-hypothesis `agent_metadata`

Illustrative MS2 RunTrace shape:

```
{
  "schema_version": "2.0-ms2",
  "retrieval_mode": "graph",
  "retrieval_context": {
    "external_memory": [],
    "contract_evidence": []
  },
  "hypothesis_traces": [
    {
      "hypothesis_id": "H06",
      "agent_metadata": {
        "agent_id": "H06",
        "rag_query": "consultant disclosure",
        "rag_hits": 5,
        "rag_mode": "graph"
      }
    }
  ]
}
```

Most importantly, the schema explicitly separates `external_memory` and `contract_evidence`. Table 3 summarizes the evidence policy boundary used throughout this report.

Final decision evidence is constrained to contract evidence shape to preserve legal grounding integrity.

Table 3: Evidence Boundary Policy

| **Source** | **Used For** | **May Be Cited as Final Legal Evidence?** |
|---|---|---|
| Retrieved `train.json` examples (`external_memory`) | Reasoning support, calibration, comparative context | No |
| Analyzed contract spans (`contract_evidence`) | Grounded legal justification and quoted evidence | Yes |
| Conversation history | Context continuity across turns | No |
| NLI labels | Classification output and routing cues | Only with supporting contract evidence |

## 11. Validation and Safety Constraints

The Milestone 2 architecture and schemas enforce the following constraints:

- Only `data/train.json` may be indexed for retrieval.
- `data/test.json` is evaluation-only and must not be indexed.
- Retrieved examples are external memory only and are not valid final legal evidence.
- Final evidence must map to spans in the analyzed contract.
- NLI outputs and trace artifacts must remain schema-valid.
- Quote integrity and groundedness checks should be preserved.
- `Qwen3-4B` orchestrator/conversation may use thinking.
- Fine-tuned `Qwen3-1.7B` NLI core must run with thinking disabled to avoid structured-output drift.

These constraints complement the evidence boundary shown in Table 3 and act as the operational safety envelope for Milestone 2 behavior.

## 12. Limitations and Future Work

Current design limitations and next steps include:

- Vector RAG and GraphRAG are separate retrieval modes, not a hybrid retrieval strategy.
- Vector retrieval depends on embedding quality and can surface semantically similar but legally weak examples.
- Graph retrieval is interpretable but constrained by manually defined concept and synonym coverage.
- Long contracts can create context-window pressure for end-to-end orchestration and conversation.

- The conversation layer must continuously avoid citing retrieved training examples as legal evidence.

Architectural and pending work items:

- Dispatcher implementation (architecturally specified).
- Hypothesis worker pool implementation (architecturally specified).
- Production aggregator implementation (architecturally specified).
- Quote repair and post-processing improvements.
- Hybrid retrieval research (vector + graph orchestration).
- UI/API integration beyond current CLI-oriented interfaces.

## 13. Implemented vs Architectural / MS3

The implementation split below keeps status statements file-truthful while distinguishing present artifacts from planned MS3 orchestration components.

Table 4: Implementation Status Summary

| Artifact | Current Status | Notes |
|---|---|---|
| `architecture/architecture.yaml` | Present | Living architecture specification. |
| `architecture/architecture.pdf` | Present | Diagram deliverable from architecture spec. |
| `src/types.py` | Present | Shared Milestone 2 TypedDict contracts. |
| `src/model_loader.py` | Present | Device and model-loading logic. |
| `src/conversation_agent.py`, `agent.py` | Present | Conversation flow and CLI entry point available. |
| `src/rag_vector.py` | Present | Vector RAG pipeline with FAISS index and re-ranking. |
| `src/rag_graph.py` | Present | GraphRAG pipeline with concept/hypothesis-targeted retrieval. |
| `pipeline/03_build_index.py` | Present | Shared indexing entry point (`--mode vector\|graph\|all`). |
| Dispatcher / worker pool / aggregator modules | Not present | Architectural/MS3-target orchestration components. |

As reflected in Table 4, Vector RAG, GraphRAG, and the shared indexing entry point are fully implemented, while full dispatcher/worker/aggregator orchestration remains MS3-target.

## 14. Conclusion

Milestone 2 transforms ContractLens from a batch legal-NLI evaluator into an agentic, conversational NDA review system that combines a `Qwen3-4B` orchestrator, a fine-tuned `Qwen3-1.7B` NLI core, vector retrieval, graph retrieval, hypothesis-level decomposition, an updated playbook, and `RunTrace 2.0` auditability.

The central design principle remains strict separation between `external_memory` and final contract-grounded legal evidence via `contract_evidence`. This principle preserves legal traceability and

prevents retrieval support artifacts from being misrepresented as contractual facts.